9 spec motion cho LIME: đặc tả để implement ngay

Chi tiết hoá 9 spec từ báo cáo nghiên cứu 3 repo của feitangyuan (onetake, motion-web, doodle-anim) thành đặc tả kỹ thuật đầy đủ: schema Zod, contract cho comp HTML, ngưỡng đo của Oracle QA, tham số render và checklist định nghĩa hoàn thành cho từng spec.

> **Cách dùng tài liệu này.** Mỗi spec là một đơn vị implement độc lập: đọc mục tiêu và phạm vi, làm theo các mục kỹ thuật đánh số, và chỉ đóng spec khi toàn bộ checklist “Định nghĩa hoàn thành” ở cuối spec đạt. Thứ tự ưu tiên: P0 = Spec A, B; P1 = Spec C, G, H; P2 = Spec D, E, F, I (chi tiết ở Phụ lục C).

Hệ thống nền: pipeline 6 phase trong `orchestrator.ts`; job qua BullMQ / Redis; API Fastify; tiến độ đẩy về UI qua SSE (Server-Sent Events); file tạm ở `tmp/<jobId>/`. Tài liệu dùng nhất quán các tên này. Phase 4.5 (Oracle QA, Spec C) là phase mới duy nhất được đề xuất chèn thêm.

Phiên bản: 1.0 — Ngày: 14/10/2026

Đối tượng đọc: team phát triển LIME (backend, render worker, frontend studio)

Tài liệu tiền thân: “Báo cáo 3 repo motion & spec LIME” (14/10/2026)

Mục lục

Mục	Phase tác động	Ưu tiên
0. Tổng quan, phạm vi và thuật ngữ	Toàn pipeline	—
1. Spec A — Motion Constitution + carry plan	Phase 1	P0
2. Spec B — Deterministic Rendering Contract	Phase 3, 5	P0
3. Spec C — Oracle QA (phase 4.5 mới)	Phase 4.5	P1
4. Spec D — Shutter motion blur	Phase 5	P2
5. Spec E — Sound from events	Phase 3, 6	P2
6. Spec F — Nhân vật procedural (port MIT)	Phase 1, 2, 3	P2
7. Spec G — Karaoke subtitle + dọc 9:16	Phase 1, 2, 3, 5	P1
8. Spec H — Font subsetting, comp tự chứa	Phase 3	P1
9. Spec I — Template authoring spec	Quy trình template	P2
Phụ lục A–D: tra nhanh, license, roadmap, nguồn	—	—

0. Tổng quan

0.1. Mục tiêu

Tài liệu này biến 9 phác thảo trong báo cáo nghiên cứu thành đặc tả đủ chi tiết để implement mà không cần quay lại đoán ý: mỗi quyết định kỹ thuật đều có giá trị cụ thể (schema, enum, ngưỡng, tham số), mỗi spec đều có test chấp nhận và định nghĩa hoàn thành. Bốn khoảng trống tài liệu nhắm tới: (1) kịch bản thiếu thiết kế ở boundary nên video đọc như slideshow; (2) chuyển động thiếu từ vựng chuẩn nên mỗi video một “giọng”; (3) chưa có tầng kiểm tra tự động trước render full; (4) tính deterministic chưa thành contract nên render song song và render lại chưa có gì bảo chứng.

0.2. Phạm vi và ngoài phạm vi

Trong phạm vi: thay đổi ở `orchestrator.ts` và các module phase 1–6, schema kịch bản, contract của comp HTML, render worker, QA, âm thanh ở phase 6, và quy trình đóng góp template. **Ngoài phạm vi:** thay đổi mô hình LLM đang dùng, thay đổi hạ tầng BullMQ/Redis/Fastify, giao diện người dùng ngoài các điểm hiển thị QA report (Spec C) và badge template (Spec I), và mọi nội dung lấy nguyên văn từ onetake/motion-web (xem Phụ lục B về license).

0.3. Pipeline sau khi áp dụng 9 spec

```
Prompt -> [Phase 1] LLM sinh kich ban JSON
            + boundaries[] + motion tokens (Spec A)
          -> [Phase 2] TTS tung scene; BeatSync do duration
            + word-level timing (Spec G), nhan vat (Spec F)
          -> [Phase 3] Rap comp HTML doc lap (GSAP)
            + deterministic contract (Spec B)
            + __events() (Spec E) + font subset (Spec H)
          -> [Phase 4] Preview snapshot WebP tung scene
          -> [Phase 4.5 MOI] Oracle QA - 5 legs (Spec C)
            fail -> reroll Phase 1 (toi da 2 lan)
            warn -> cho user duyet tren UI
          -> [Phase 5] Render Playwright + __seekTo(t)
            + shutter blur tuy chon (Spec D)
            + viewport 16:9 hoac 9:16 (Spec G)
          -> [Phase 6] Mux MP4: ffmpeg mix TTS + BGM + SFX
            master -16 LUFS (Spec E)
File tam: tmp/<jobId>/ | Job: BullMQ/Redis | Tien do: SSE
```

Sơ đồ trên cố ý viết không dấu để an toàn trong khối monospace; tên phase và tên hàm giữ nguyên như trong code.

0.4. Bảng thuật ngữ

Thuật ngữ	Định nghĩa dùng trong tài liệu này
Comp	File HTML độc lập của một job, chứa toàn bộ timeline GSAP; đơn vị mà Phase 3 ráp và Phase 5 render.
Boundary	Điểm chuyển giữa hai scene liên tiếp. Với N scene có đúng N−1 boundary.
Carry	Yếu tố hình ảnh của scene trước sống sót qua boundary và biến thành một phần của scene sau (xem Spec A).
Carry score	Tỉ lệ boundary đạt kiểm tra continuity trong Oracle QA: số boundary đạt / tổng số boundary (Spec C).
Mechanism	Cơ chế biến đổi của carry: morph, zoomThrough, iris, gather, handoff, hardCut (Spec A).
Near-still	Đoạn gần-tĩnh: khoảng thời gian chuyển động trên màn hình rất thấp, để nhịp video có điểm “đáp”. Ngưỡng định lượng ở Spec A/C.
t80	Thời điểm một chuyển động đạt 80% quãng đường của nó; chỉ số mô tả hình dạng đường cong tốc độ (bật nhanh hay chữ S). Dùng khi phân tích curve, không bắt buộc trong schema v1.
Motion token	Giá trị chuyển động có tên và ý định rõ (duration, easing, stagger), LLM chỉ được chọn token, không tự chế số (Spec A).
Deterministic contract	Tập quy ước buộc cùng một input (job, seed, thời điểm t) luôn cho cùng một frame (Spec B).
Shutter (màn trập)	Khoảng thời gian “mở” quanh thời điểm của một frame khi lấy mẫu motion blur (Spec D).
LUFS / dBTP	Đơn vị độ to tích hợp (LUFS) và đỉnh thật (dB True Peak) dùng khi master âm thanh (Spec E).
BeatSync	Cơ chế hiện có của LIME: duration audio TTS đo được ở Phase 2 là “sự thật” để căn thời lượng animation.
Reroll	Sinh lại Phase 1 (kịch bản/comp) với seed hoặc feedback khác sau khi Oracle QA fail.

1. Spec A — Motion Constitution + carry plan P0

Phase tác động: Phase 1 (LLM sinh kịch bản) và system prompt. **Mục tiêu:** loại cảm giác slideshow ngay từ kịch bản và chuẩn hoá "giọng" chuyển động cho mọi video. Spec gồm bốn phần: schema `boundaries[]` bắt buộc (1.1), ràng buộc rhythm kiểm chứng được (1.2), bộ motion tokens của LIME (1.3) và luật anti-slop trong system prompt (1.4). Phần 1.5 là ví dụ JSON trọn vẹn.

1.1. Mở rộng schema kịch bản: boundaries[]

Mỗi boundary khai báo phần tử nào được mang qua (`carry.element` — dùng đúng `id` của phần tử trong comp để Spec C truy vấn được bằng DOM) và cơ chế biến đổi (`carry.mechanism`). Enum mechanism cố định ở v1; thêm mechanism mới phải qua review vì Oracle và thư viện move ở Phase 3 đều phụ thuộc enum này.

Mechanism	Ý nghĩa và khi dùng
`morph`	Phần tử đổi hình dạng/vị trí thành phần tử của scene sau (thẻ phình thành trang, prompt bar mở thành cửa sổ).
`zoomThrough`	Camera xuyên qua phần tử; phần tử lấp đầy khung hình rồi trở thành bối cảnh scene sau.
`iris`	Mở/đóng hình tròn (hoặc mask tương đương) quanh phần tử carry, lộ scene sau.
`gather`	Nhiều phần tử tụ lại thành một khối rồi khối đó mở thành scene sau.
`handoff`	Phần tử được "trao tay": di chuyển liên tục sang vị trí/vai trò mới trong scene sau, không biến mất giữa chừng.
`hardCut`	Cắt thẳng, không carry. **Chỉ** được dùng cho nhịp burst (dồn dập có chủ đích) hoặc endcard; mọi lần dùng phải có trường `reason` giải thích. Oracle QA tính hardCut là boundary không đạt continuity, trừ khi `reason` thuộc danh sách cho phép.

```
// script.schema.ts — mo rong schema hien co (Zod)
const mechanismSchema = z.enum([
  "morph", "zoomThrough", "iris",
  "gather", "handoff", "hardCut",
]);
const boundarySchema = z.object({
  id: z.string(),               // vd "b1"
  fromScene: z.string(),        // sceneId
  toScene: z.string(),          // sceneId
  carry: z.object({
    element: z.string(),        // element id trong comp
    mechanism: mechanismSchema,
    reason: z.string().optional(), // bat buoc neu hardCut
  }),
});
```

1.2. Ràng buộc rhythm (kiểm chứng bằng số, không bằng cảm tính)

Ràng buộc	Ngưỡng v1	Kiểm chứng ở đâu
Tỉ số shot dài nhất / ngắn nhất	Tối thiểu 4 (max/min ≥ 4)	Zod refine ở Phase 1 + leg rhythm của Oracle
Đoạn near-still	Ít nhất 1 đoạn ≥ 1.5s có chuyển động rất thấp	Khai báo trong kịch bản; Oracle đo lại trên stills
Cấm uniform cadence	Hệ số biến thiên CV = độ lệch chuẩn / trung bình của độ dài shot phải ≥ 0.3	Zod refine + Oracle (fail nếu CV < 0.3)

Ba ràng buộc dùng chung một hàm kiểm tra duy nhất (`checkRhythm(durations)`) để Phase 1 và Oracle không bao giờ lệch ngưỡng. Zod refine cross-field bổ sung hai điều kiện toàn cục: (a) tổng duration các scene phải khớp tổng duration BeatSync trong dung sai ±0.1s sau khi Phase 2 có số đo thật — ở Phase 1 kiểm tra trên duration dự kiến, ở cuối Phase 2 kiểm tra lại và fail job sớm nếu lệch; (b) `boundaries.length === scenes.length − 1` và chuỗi from/to phải liên tục (boundary i nối scene i với scene i+1).

```
const scriptSchema = baseScriptSchema
 .extend({ boundaries: z.array(boundarySchema) })
 .refine(s => s.boundaries.length === s.scenes.length - 1,
   { message: "boundaries phai bang scenes - 1" })
 .refine(s => checkRhythm(s.scenes.map(x => x.duration)).ok,
   { message: "rhythm: max/min >= 4, CV >= 0.3, co near-still" })
 .refine(s => s.boundaries.every(b =>
   b.carry.mechanism !== "hardCut" || !!b.carry.reason),
   { message: "hardCut bat buoc co reason" });
```

1.3. Motion tokens của LIME (LLM chỉ chọn token, không tự chế số)

Token duration	Giá trị	Ý định (intent)
`dur.micro`	150–200 ms	Phản hồi nhỏ: hover, nhấn, tick
`dur.fast`	250–350 ms	Phần tử phụ xuất hiện/rời đi
`dur.base`	450–600 ms	Chuyển động mặc định của scene
`dur.slow`	700–900 ms	Nhấn mạnh, chuyển trọng tâm
`dur.cinematic`	1000–1400 ms	Cold-open, camera move lớn, outro

Quy tắc áp dụng ghi vào prompt: “Không giải thích được duration bằng intent thì giảm nó một nửa.” Duration ngoài 5 token trên là lỗi schema.

Token easing	GSAP tương ứng	Dùng khi
`ease.enter`	`expo.out`	Phần tử vào khung: nhanh đầu, đáp mềm
`ease.pop`	`back.out(1.7)`	Xuất hiện cần overshoot nhẹ (badge, thẻ)
`ease.playful`	`elastic.out(1, 0.5)`	Điểm nhấn vui, ký hiệu cảm xúc; tối đa 1 lần/scene
`ease.camera`	`circ.out`	Camera pan/zoom, di chuyển dài
`ease.flow`	`sine.inOut`	Chuyển động nền liên tục, dao động nhẹ

Pattern stagger	Bước nhảy	Dùng khi
`stagger.grid`	0.04s	Lưới nhiều phần tử (icon, ô, chữ theo lưới)
`stagger.list`	0.08s	Danh sách, checklist, dòng chữ lần lượt
`stagger.dramatic`	0.15s	Dồn dập có kịch tính; ít phần tử, cần đọc từng cái

1.4. Luật anti-slop — đoạn bắt buộc trong system prompt

1. Cấm fade `opacity` 0→1 làm chuyển động duy nhất của một phần tử; mọi lần xuất hiện phải kèm transform (vị trí, tỉ lệ) hoặc mask.
2. Cấm easing tuyến tính (`linear` / `none` trong GSAP) cho mọi chuyển động trừ thanh tiến trình và chuyển động nền tốc độ hằng số có lý do ghi rõ.
3. Mọi move phải có damping hoặc overshoot: hoặc dùng token easing ở bảng 1.3, hoặc khai báo spring; cấm tween “đi đều rồi dừng đột ngột”.
4. Cấm hai scene liên tiếp dùng cùng một cơ chế vào cảnh (entrance) — sự lặp lại là dấu hiệu slideshow.
5. Mọi duration/easing/stagger phải tham chiếu token ở mục 1.3; số tự chế ngoài token là lỗi, Phase 1 phải reject và retry.

1.5. Ví dụ JSON đầy đủ — video 3 scene

```
{
  "format": "16:9", "seed": 104729,
  "scenes": [
    { "id": "s1", "duration": 2.0, "intent": "hook",
      "voiceText": "Y tuong cua ban, thanh video.",
      "moves": [{ "el": "promptBar", "token": "dur.cinematic", "ease": "ease.enter" }] },
    { "id": "s2", "duration": 8.0, "intent": "product",
      "voiceText": "Go prompt, LIME dung timeline...",
      "nearStill": { "from": 5.0, "to": 6.6 },
      "moves": [{ "el": "editor", "token": "dur.base", "ease": "ease.camera", "stagger":
"stagger.list" }] },
    { "id": "s3", "duration": 2.5, "intent": "outro",
      "voiceText": "LIME is motion, exclusive.",
      "moves": [{ "el": "logo", "token": "dur.slow", "ease": "ease.pop" }] }
  ],
  "boundaries": [
    { "id": "b1", "fromScene": "s1", "toScene": "s2", "carry": { "element": "promptBar",
"mechanism": "morph" } },
    { "id": "b2", "fromScene": "s2", "toScene": "s3", "carry": { "element": "logo", "mechanism":
"handoff" } }
  ]
}
```

1. Spec A (tiếp) — kiểm tra ví dụ và hoàn thành

Kiểm tra nhanh ví dụ: durations 2.0 / 8.0 / 2.5 → max/min = 4.0 (đạt ≥ 4); CV ≈ 0.62 (đạt ≥ 0.3); có near-still 1.6s ở s2; boundaries = 2 = scenes − 1; không có hardCut.

Định nghĩa hoàn thành — Spec A

- Schema có `boundaries[]` bắt buộc, enum mechanism đủ 6 giá trị, `reason` bắt buộc khi hardCut.
- `checkRhythm()` dùng chung cho Zod refine và Oracle; 3 ngưỡng (≥4, ≥1.5s, CV ≥ 0.3) có unit test.
- Bảng token duration/easing/stagger nằm trong prompt và trong validator; output dùng số ngoài token bị reject.
- 5 luật anti-slop có mặt nguyên văn trong system prompt production.
- Ví dụ 3 scene ở mục 1.5 validate pass và render được end-to-end.

Ghi chú triển khai Spec A

Khi retry Phase 1 do validate fail, thông báo lỗi của Zod phải được đưa nguyên văn vào prompt retry để LLM sửa đúng trường, thay vì sinh lại toàn bộ kịch bản từ đầu. Giới hạn retry hiện tại của Phase 1 giữ nguyên; Spec A không tăng số lần retry, chỉ làm mỗi lần retry có thông tin tốt hơn. Motion tokens sau khi chốt ở v1 chỉ được bổ sung, không đổi giá trị của token cũ — đổi giá trị sẽ làm video cũ và mới lệch “giọng” và làm vô hiệu các verify criteria của Spec I đã viết theo token cũ.

2. Spec B — Deterministic Rendering Contract P0

Phase tác động: Phase 3 (ráp comp, kiểm tra tĩnh) và Phase 5 (render). **Mục tiêu:** cùng một job, cùng một thời điểm t, ở bất kỳ worker nào, lần render nào, cũng cho ra cùng một frame — điều kiện để render song song, retry frame và render lại job cũ đều an toàn. Ý tưởng contract lấy từ doodle-anim (MIT, được port — xem Spec F và Phụ lục B); phần cấm và kiểm tra dưới đây là cài đặt riêng của LIME.

2.1. Năm quy ước của contract

1. **Mọi randomness đến từ PRNG có seed.** Thuật toán chuẩn: mulberry32. Seed gốc của scene: `seed = hash(jobId + sceneId)` (hash nguyên 32-bit ổn định, ví dụ FNV-1a hoặc xxhash32; chọn một và cố định trong code, không đổi giữa các phiên bản nếu muốn job cũ render lại khớp).
2. **Seed theo element id.** Mỗi phần tử/hiệu ứng tự suy seed riêng: `elementSeed = hash(sceneSeed + elementId)`. Thêm/bớt một phần tử không làm các phần tử khác đổi hành vi ngẫu nhiên.
3. **Mọi animation là pure function của t.** Trạng thái tại thời điểm t chỉ phụ thuộc vào t và dữ liệu comp; không phụ thuộc thứ tự frame đã render, số lần seek, hay trạng thái tích luỹ. GSAP timeline phải được seek tuyệt đối, không dùng `play()` theo thời gian thực trên render path.
4. **Comp expose giao diện seek và seed.** `window.__seekTo(t)` (đã có ở Phase 5) là chuẩn bắt buộc; bổ sung `window.__renderSeed` để worker và QA đọc lại seed đang dùng, phục vụ debug và test khớp frame.
5. **Alpha blending dạng nhân cho cross-fade.** Khi hai lớp chồng nhau trong chuyển cảnh, comp dùng phép trộn nhân (multiply-style) theo quy ước chung thay vì opacity tuyến tính mặc định, để kết quả chồng lớp không phụ thuộc thứ tự render từng lớp.

2.2. Quy định cho code LLM sinh ra (Phase 3 kiểm tra tĩnh)

Cấm trên render path	Lý do
`Math.random()`	Không seed → frame không lặp lại được
`Date.now()`, `new Date()`	Phụ thuộc đồng hồ hệ thống → trạng thái không còn là f(t)
`performance.now()`, `requestAnimationFrame` tự chạy	Phụ thuộc thời gian thực và nhịp máy render

Thay thế duy nhất được phép: helper `RNG(seed)` do LIME inject vào mọi comp. Phase 3 chạy bước lint (quét AST hoặc regex trên bản dựng cuối) trước khi lưu comp; vi phạm → reject comp, trả feedback cho bước sinh code, không chờ tới Phase 5 mới phát hiện.

```
// rng.ts — helper inject vao moi comp (mulberry32)
export function RNG(seed: number) {
  let a = seed >>> 0;
  return function next(): number {
    a = (a + 0x6D2B79F5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
// Trong comp: const rand = RNG(hash(jobId + sceneId + elId));
```

2.3. Ảnh hưởng lên Phase 5 và vận hành

- **Render song song an toàn:** nhiều scene (và nhiều frame trong một scene) render đồng thời trên các worker khác nhau cho kết quả như render tuần tự, vì không frame nào phụ thuộc frame trước.
- **Retry frame:** frame lỗi (timeout, crash tab) render lại cho kết quả giống hệt frame gốc dự kiến; cơ chế "lỗi thì dùng lại frame trước" hiện tại chỉ còn là phương án cuối, không phải mặc định im lặng.
- **Render lại job cũ:** seed lưu cùng job trong DB; mở lại job sau nhiều tháng vẫn tái tạo đúng frame nếu contract và hash không đổi. Mọi thay đổi làm lệch kết quả (đổi thuật toán hash, đổi PRNG) phải tăng version của contract và ghi vào metadata job.
- **Cache:** frame có thể cache theo khoá (jobId, sceneId, t, contractVersion) — chỉ hợp lệ khi contract được tuân thủ.

2.4. Test chấp nhận

1. Render cùng một job 2 lần độc lập (2 tiến trình worker khác nhau), trích toàn bộ frame của một scene mẫu, so `sha256` từng frame: phải khớp 100%.
2. Render một scene theo thứ tự frame xáo trộn (ví dụ lẻ trước, chẵn sau): frame tại cùng t phải khớp bản render tuần tự.
3. Comp chứa `Math.random()` trên render path: Phase 3 phải reject kèm thông báo đúng vị trí vi phạm.
4. Đọc `window.__renderSeed` từ comp đã ráp: phải bằng seed suy từ (jobId, sceneId) theo hàm hash đã cố định.

> **Lưu ý thực tế:** khớp sha256 đòi hỏi cả môi trường render ổn định (cùng phiên bản Chromium, cùng font — xem Spec H). Nếu frame lệch ở mức pixel do anti-aliasing giữa hai máy khác cấu hình, ghi nhận là vấn đề môi trường, không phải vi phạm contract; test chấp nhận chạy trên cùng pool worker production.

Định nghĩa hoàn thành — Spec B

- Contract 5 điều được ghi trong tài liệu nội bộ của comp và trong prompt sinh code Phase 3.
- Helper `RNG(seed)` + hàm `hash()` cố định đã merge; seed lưu cùng job trong DB.
- Lint Phase 3 chặn được cả 3 mẫu cấm; comp vi phạm không tới được Phase 5.
- `window.__renderSeed` đọc được từ mọi comp mới; test sha256 2 lần render khớp 100% chạy trong CI.

3. Spec C — Oracle QA, phase 4.5 mới P1

Phase tác động: chèn Phase 4.5 giữa preview (Phase 4) và render full (Phase 5). **Mục tiêu:** chặn video kém chất lượng hoặc lỗi kỹ thuật trước khâu render tốn kém nhất, bằng các phép đo tự động có ngưỡng rõ ràng — ý tưởng "oracle" học từ onetake và tự cài đặt lại cho LIME (Phụ lục B).

3.1. Kiến trúc: mở rộng từ preview snapshot, không dựng hệ mới

Phase 4 hiện chụp snapshot WebP từng scene. Oracle tái dùng đúng phiên Playwright đó, mở rộng lịch chụp: (a) tại mỗi boundary, chụp still tại `t − ε` và `t + ε` với ε = 0.15s (đủ gần để phần tử carry chưa kịp rời vị trí, đủ xa để chuyển đổi đã bắt đầu); (b) chụp sample đều trong từng scene (mặc định mỗi 0.5s, tối thiểu 3 still/scene) phục vụ legs rhythm, curves và framing; (c) truy vấn DOM trực tiếp trong trang (`getBoundingClientRect()`, kiểm tra tồn tại của phần tử) thay vì chỉ phân tích ảnh — rẻ và chính xác hơn cho continuity. Toàn bộ kết quả ghi thành QA report (mục 3.4). Tiến độ Phase 4.5 đẩy về UI qua SSE như các phase khác; trạng thái job thêm `qaStatus: pass | warn | fail`.

```
Phase 4 preview WebP
   |
   v
[4.5] Oracle: stills boundary +/-0.15s + sample 0.5s
   |-- leg rhythm      --|
   |-- leg continuity  --|
   |-- leg curves      --+--> QA report JSON (DB + UI)
   |-- leg framing     --|
   |-- leg audio       --|
   |
   +-- FAIL -> reroll Phase 1 (max 2, kem feedback)
   +-- WARN -> cho user duyet tren UI
   +-- PASS -> Phase 5 render full
```

3.2. Năm legs và ngưỡng (phần 1: rhythm, continuity)

Leg	Cách đo	Ngưỡng
Rhythm	CV của độ dài các shot (dùng chung `checkRhythm()` với Spec A) + quét sample tìm đoạn near-still	**Fail** nếu CV < 0.3. **Warn** nếu không có đoạn tĩnh ≥ 1.5s (không fail, vì có video ngắn hợp lệ không có đoạn tĩnh)
Continuity	Với mỗi boundary: tại t−ε và t+ε, query DOM phần tử `carry.element`; đạt khi phần tử tồn tại ở cả hai phía *và* bounding box có overlap, hoặc tâm điểm di chuyển không quá 40% đường chéo viewport giữa hai still (chuyển tiếp hợp lý, không "nhảy" mất hút)	Carry score = boundary đạt / tổng boundary. **Fail** < 0.5; **warn** < 0.75; pass ≥ 0.75. `hardCut` có `reason` hợp lệ được loại khỏi mẫu số

Vì sao đo bằng DOM mà không bằng đối sánh ảnh: LIME biết chính xác `element id` từ schema Spec A, nên kiểm tra tồn tại + hình học của đúng phần tử đó là phép đo trực tiếp của định nghĩa carry. Đối sánh ảnh giữ làm phương án dự phòng cho template cũ chưa có element id (Spec I sẽ chuẩn hoá dần).

3.2. Năm legs và ngưỡng (phần 2: curves, framing, audio)

Leg	Cách đo	Ngưỡng
Curves	Từ chuỗi bounding box của phần tử chính qua các still sample: displacement lớn nhất giữa hai sample kề, quy về px/frame tại fps đích	**Fail** nếu displacement > 80 px/frame mà Spec D (shutter blur) chưa bật cho job. Khi Spec D bật, leg này chuyển thành **warn** để theo dõi, không chặn
Framing	Subject khai báo trong kịch bản (trường `subject` của scene) phải có bounding box giao với viewport ở mọi still sample	**Fail** nếu subject rời viewport hoàn toàn ở bất kỳ sample nào mà kịch bản không khai báo đó là exit có chủ đích
Audio	Phân tích track TTS + BGM dự kiến sau Phase 2: peak và cửa sổ RMS trượt 0.5s	**Fail** nếu peak ≥ −3 dBFS (nguy cơ clipping). **Warn** nếu không có đoạn quiet (RMS < −40 dB kéo dài ≥ 1s) — video nói liên tục không nghỉ nghe rất mệt

3.3. Hành động khi fail / warn

- **Fail → auto-reroll Phase 1, tối đa 2 lần.** Feedback đưa vào prompt reroll phải cụ thể theo leg, ví dụ: “Boundary b2: phần tử ‘logo’ không tồn tại tại t+ε; hãy giữ logo trong DOM xuyên boundary hoặc đổi mechanism sang handoff.” Reroll dùng seed mới và giữ nguyên prompt gốc của user. Sau 2 lần vẫn fail: dừng ở trạng thái `qa_failed`, hiển thị lý do cho user, không render full trong im lặng.
- **Warn → gắn cờ, chờ user duyệt trên UI.** Job không tự đi tiếp; UI hiển thị danh sách warn kèm still minh hoạ, user bấm duyệt mới sang Phase 5. Warn không reroll tự động.
- **Pass → Phase 5** ngay, QA report vẫn lưu để truy vết.

3.4. QA report JSON (lưu DB + hiển thị UI)

```
{
  "jobId": "job_123", "qaStatus": "warn", "attempt": 1,
  "legs": {
    "rhythm":    { "status": "pass", "cv": 0.62 },
    "continuity":{ "status": "warn", "carryScore": 0.67,
                   "failed": ["b2"] },
    "curves":    { "status": "pass", "maxPxPerFrame": 42 },
    "framing":   { "status": "pass" },
    "audio":     { "status": "pass", "peakDbfs": -6.1,
                   "quietSec": 1.8 }
  },
  "createdAt": "2026-10-14T00:00:00.000Z"
}
```

Định nghĩa hoàn thành — Spec C

- Phase 4.5 chạy trong `orchestrator.ts`, có trạng thái SSE và `qaStatus` trong DB.
- Cả 5 legs có unit/integration test với comp mẫu đạt và comp mẫu fail từng leg.
- Reroll tối đa 2 lần kèm feedback theo leg; lần 3 fail dừng ở `qa_failed` kèm lý do hiển thị.
- QA report JSON đúng mẫu 3.4, xem được trên UI cùng still minh hoạ cho warn/fail.

4. Spec D — Shutter motion blur P2

Phase tác động: Phase 5 (render worker). **Mục tiêu:** chuyển động nhanh không còn “khựng từng frame” mà nhoè theo hướng di chuyển như quay phim thật. Ý tưởng học từ onetake, cài đặt lại trong vòng lặp screenshot hiện có.

4.1. Contract phía comp: window.__motion(t0, t1)

Comp khai báo hàm `window.__motion(t0, t1)` trả về quãng di chuyển lớn nhất (pixel travel, tính theo phần tử di chuyển xa nhất trên viewport) trong khoảng [t0, t1]. Cài đặt phía comp có thể suy từ GSAP timeline (vị trí đầu/cuối của các tween đang chạy) — renderer không tự đoán. Comp cũ chưa có `__motion` thì coi như travel = 0 (không blur, hành vi như hiện tại).

4.2. Thuật toán lấy mẫu trong render worker

1. Với frame tại thời điểm t, gọi `__motion(t − shutter/2, t + shutter/2)`; shutter mặc định = 1/fps (mở toàn bộ khoảng frame) và cấu hình được.
2. Nếu travel ≤ ngưỡng `gapPx` (mặc định 2 px): chụp 1 frame như hiện tại — bỏ qua đa mẫu khi hold/tĩnh để không tốn chi phí vô ích.
3. Nếu travel > `gapPx`: lấy K mẫu đều trong khoảng shutter, với K kẹp trong [`samplesMin`, `samplesMax`] và tỉ lệ với travel (gợi ý: K = clamp(round(travel / 12), samplesMin, samplesMax); mặc định samplesMin = 3, samplesMax = 9). Với mỗi mẫu: `__seekTo(t_k)` rồi screenshot.
4. **Trung bình trong linear light:** chuyển từng ảnh mẫu từ sRGB sang linear, cộng trung bình theo pixel, chuyển ngược về sRGB rồi mới encode JPEG q90 như pipeline hiện tại. Cộng trung bình trực tiếp trên giá trị sRGB cho blur tối và sai màu — đây là lỗi phải tránh, có test riêng.

```
// render worker — vong lap frame (rut gon)
const travel = await page.evaluate(
  ([a, b]) => window.__motion?.(a, b) ?? 0,
  [t - shutter / 2, t + shutter / 2]);
if (travel <= cfg.gapPx) {
  frame = await shotAt(t);              // 1 mau nhu hien tai
} else {
  const K = clamp(Math.round(travel / 12), cfg.samplesMin, cfg.samplesMax);
  const samples = [];
  for (let k = 0; k < K; k++)
    samples.push(await shotAt(t - shutter/2 + shutter*(k+0.5)/K));
  frame = averageInLinearLight(samples); // sRGB->linear->avg->sRGB
}
```

4.3. Chi phí và cấu hình

Tham số	Mặc định	Ghi chú
`blur.enabled`	`false` (draft) / `true` (final)	Bật cho bản final; draft tắt để giữ tốc độ preview
`blur.samplesMin / Max`	3 / 9	K mẫu tối thiểu/tối đa cho frame có chuyển động
`blur.gapPx`	2 px	Ngưỡng travel coi như tĩnh
`blur.shutter`	1/fps	Khoảng mở màn trập tính bằng giây

4. Spec D (tiếp) — chi phí và hoàn thành

Ước tính chi phí (phải đo lại trên pool thật): frame chuyển động tốn xấp xỉ K lần phần screenshot; frame tĩnh không tăng. Với khoảng 40% frame chuyển động và K trung bình 5, Phase 5 dự kiến tăng khoảng 2–3 lần cho bản final. Vì vậy Spec D không bật cho draft/preview, và leg curves của Oracle chỉ fail khi blur chưa bật — hai spec đọc cùng một cờ cấu hình.

Định nghĩa hoàn thành — Spec D

- `__motion(t0, t1)` có trong tài liệu contract; comp mẫu trả travel đúng với tween đã biết.
- Frame tĩnh chỉ chụp 1 mẫu (có log/metric chứng minh); frame nhanh chụp K mẫu trong [min, max].
- Test chứng minh average thực hiện trong linear light (ảnh mẫu tổng hợp cho kết quả số đúng kỳ vọng).
- Cờ bật/tắt theo loại bản render (draft/final) hoạt động; thời gian render final được đo và ghi vào tài liệu vận hành.

Ghi chú vận hành

Ghi lại tỉ lệ frame phải đa mẫu và K trung bình theo từng job vào log render: hai con số này là cơ sở để hiệu chỉnh `samplesMax` và quyết định có bật blur mặc định cho mọi bản final hay chỉ cho video có chuyển động nhanh. Nếu thời gian render final vượt ngân sách, giảm `samplesMax` trước, không tắt hẳn blur — blur với K nhỏ vẫn tốt hơn không blur đối với chuyển động trên 80 px/frame.

5. Spec E — Sound from events P2

Phase tác động: Phase 3 (comp expose sự kiện) và Phase 6 (mix/master trong ffmpeg). **Mục tiêu:** hiệu ứng âm thanh (SFX) khớp đúng hành động trên màn hình thay vì đặt tay, mọi âm thanh nghe “cùng một phòng”, và bản master đạt chuẩn độ to trước khi tới tay user. Ý tưởng học từ onetake, cài đặt lại.

5.1. Contract phía comp: window.__events()

```
// Tra ve toan bo su kien am thanh cua timeline
window.__events = () => [
  { t: 0.40, type: "appear",  intensity: 0.7, pan: 0.0 },
  { t: 2.05, type: "impact",  intensity: 1.0, pan: -0.2 },
  { t: 8.00, type: "whoosh",  intensity: 0.5, pan: 0.3 },
];
// t: giay; type: nhom su kien; intensity: 0..1;
// pan: -1 (trai) .. 1 (phai)
```

Sự kiện do timeline GSAP phát ra tại đúng thời điểm hành động (xuất hiện, va chạm, chuyển cảnh...). Phase 3 kiểm tra tĩnh: `__events()` trả mảng hợp lệ, `t` tăng dần và nằm trong tổng thời lượng comp.

5.2. Sound engine: 5 material, 1 reverb

Material	Tính chất và dùng cho
`air`	Nhẹ, thoáng: whoosh, chuyển cảnh, camera move
`wood`	Trầm, ngắn: chạm, đặt xuống, đóng/mở
`glass`	Sáng, vang mảnh: xuất hiện UI, điểm nhấn, thông báo
`sub`	Rất trầm, lực: impact lớn, hạ cánh, kết thúc đoạn
`bubble`	Tròn, vui: pop, ký hiệu cảm xúc, tương tác nhẹ

Mỗi `type` sự kiện map sang một material và một preset tổng hợp (synth) có tham số theo `intensity` và `pan`. Toàn bộ SFX đi qua **một reverb duy nhất** (cùng impulse/preset cho cả job) — đây là chi tiết tạo cảm giác “cùng một phòng”; tuyệt đối không để mỗi SFX mang reverb riêng. Kho SFX tổng hợp phải tự xây hoặc dùng nguồn có license rõ ràng; không lấy assets âm thanh từ onetake/motion-web.

5.3. Mix và master ở Phase 6

- **Ducking:** BGM tự hạ dưới các hit — sidechain theo events: tại event `intensity` cao, BGM giảm (gợi ý −6 đến −10 dB) rồi hồi phục mượt trong 200–400 ms. TTS không bị duck bởi SFX; SFX phải nhường TTS.
- **ffmpeg mix:** ba bus TTS + BGM (đã duck) + SFX (đã qua reverb) trộn thành một track, encode AAC 192k như hiện tại, kèm `faststart`.
- **Master:** chuẩn hoá về **−16 LUFS** tích hợp, trần đỉnh thật **−3.5 dBTP** để AAC encode không gây clipping liên mẫu. Đo lại sau encode; lệch quá ±0.5 LUFS thì master lại, không xuất file.

Định nghĩa hoàn thành — Spec E

- `__events()` có validator ở Phase 3; comp mẫu trả events đúng thứ tự và đúng khoảng thời gian.
- Đủ 5 material, mỗi material có test nghe/đo phổ cơ bản; toàn bộ SFX dùng chung 1 reverb.
- Ducking hoạt động theo events (có test đo mức BGM tại thời điểm hit).
- File MP4 xuất ra đo được −16 LUFS ±0.5 và peak ≤ −3.5 dBTP.

6. Spec F — Nhân vật procedural P2

Phase tác động: Phase 1 (schema), Phase 2 (nguồn hình ảnh), Phase 3 (comp). **Mục tiêu:** hệ nhân vật/mascot sinh bằng code — không phụ thuộc kho ảnh ngoài, không vướng bản quyền ảnh — cho video explainer và kể chuyện, vốn hiện phụ thuộc ảnh Wikimedia Commons.

> **License — đọc kỹ trước khi port.** Spec này port code từ **doodle-anim (license MIT)** — repo duy nhất trong ba repo được phép dùng code trong sản phẩm thương mại. Điều kiện bắt buộc: (1) giữ thông báo bản quyền và license MIT trong thư mục code đã port; (2) ghi nguồn trong README LIME và comment đầu file đã port (tên repo, tác giả feitangyuan, URL — Phụ lục D); (3) không áp dụng cho onetake/motion-web — hai repo đó chỉ học ý tưởng (Phụ lục B).

6.1. Phạm vi port (có chọn lọc, không bê toàn bộ repo)

Thành phần	Nội dung port và cách dùng trong LIME
Seeded avatar generator	Sinh nhân vật theo seed: mặt, tóc, mắt suy deterministic từ seed. Seed: `hash(jobId)` (hàm hash của Spec B) để nhân vật nhất quán cả video và tái tạo được khi render lại
Walk cycle + bone kinematics	Hệ xương và chu kỳ bước đi; tư thế là pure function của t — bắt buộc khớp Spec B, không giữ trạng thái tích luỹ
Thư viện ký hiệu cảm xúc	Ký hiệu vẽ bằng code: trái tim, mồ hôi, giận, “zzz”, chấm than/hỏi — hiển thị cạnh nhân vật theo kịch bản
Beat actions	Bộ hành động đặt tên sẵn: `enter`, `walk`, `run`, `idle`, `emote` — đơn vị LLM gọi trong kịch bản

6.2. Tích hợp

- **Phase 1 — schema:** scene được chứa `characterActions[]`: `character` (định danh/seed), `action` (enum 5 beat actions), `at` (thời điểm trong scene), `emote` tuỳ chọn. LLM không được đặt tên action ngoài enum.
- **Phase 3 — template mới:** hệ nhân vật đóng gói thành họ template “character explainer” trong studio; comp tuân thủ Spec B (`__seekTo(t)` đặt tư thế f(t), randomness qua `RNG(seed)`).
- **Phase 2 — nguồn hình ảnh:** nhân vật procedural bổ sung, ưu tiên cho scene có `characterActions`; scene khác vẫn dùng ảnh Wikimedia/assets như hiện tại.

```
// Mo rong schema scene (Phase 1)
characterActions: z.array(z.object({
  character: z.string(),   // vd "mascot" hoac seed id
  action: z.enum(["enter","walk","run","idle","emote"]),
  at: z.number().min(0),   // giay, trong scene
  emote: z.enum(["heart","sweat","anger","zzz","wow","question"]).optional(),
})).optional()
```

Định nghĩa hoàn thành — Spec F

- Code port nằm trong thư mục riêng, có LICENSE MIT gốc + ghi nguồn trong README và đầu file.
- Schema `characterActions[]` validate được; template “character explainer” chạy end-to-end.
- Video explainer mẫu dùng nhân vật procedural render thành công qua đủ 6 phase.

7. Spec G — Karaoke subtitle + dọc 9:16 P1

Mục tiêu: phụ đề tô sáng theo từng từ đang được đọc (karaoke) để giữ mắt người xem, và thêm định dạng dọc 720×1280 (9:16) để video LIME đăng được lên TikTok / Reels / Shorts. Hai nửa độc lập, có thể implement karaoke trước.

7.1. Karaoke subtitle — word-level timing từ BeatSync

- **Phase 2:** mở rộng BeatSync từ mức scene xuống mức từ. Ưu tiên lấy word-level timing trực tiếp từ TTS engine nếu engine trả về; nếu không, chạy forced alignment (căn audio với transcript đã biết — transcript có sẵn vì chính LIME sinh lời đọc) để suy thời điểm bắt đầu/kết thúc từng từ. Kết quả lưu cùng job: `words: [{w, start, end}]` theo giây tuyệt đối trên timeline job.
- **Phase 3:** comp render mỗi từ trong một `<span>` riêng; hàm seek theo t (cùng cơ chế `__seekTo`) bật trạng thái highlight cho từ có `start ≤ t < end`, từ đã qua giữ màu phụ, từ chưa tới giữ màu nền. Highlight phải là pure function của t (Spec B) — cấm dùng timer/CSS animation tự chạy.
- **Độ chính xác chấp nhận:** lệch highlight so với audio không quá ±120 ms trên video mẫu kiểm tra tay; lệch lớn hơn thường do alignment — sửa ở Phase 2, không "bù" bằng offset tay trong comp.

7.2. Dọc 9:16 — format là dữ kiện của job, không phải bản dựng riêng

Phase	Thay đổi
Phase 1	Schema job/script thêm `format: "16:9" | "9:16"` (mặc định `"16:9"` giữ hành vi cũ). LLM nhận format trong prompt để bố cục kịch bản cho khung dọc ngay từ đầu
Phase 2	Word-level timing cho karaoke (mục 7.1); TTS không đổi theo format
Phase 3	Comp responsive theo aspect ratio: cùng một comp, layout đổi theo chiều viewport; khai báo **safe-area 9:16**: vùng trung tâm an toàn cho chữ và subject, tránh dải trên (khu vực tên/tiêu đề của nền tảng) và dải dưới (khu vực caption/nút tương tác) — định nghĩa bằng tỉ lệ phần trăm chiều cao trong code comp và dùng nhất quán cho mọi template dọc
Phase 5	Viewport Playwright và kích thước frame theo format: 16:9 dùng preset hiện tại; 9:16 dùng **720×1280**. ffmpeg giữ nguyên chuỗi encode (libx264, AAC 192k, faststart)

Subject và framing ở bản dọc phải qua được leg framing của Oracle (Spec C) với viewport dọc — không chấp nhận bản dọc chỉ là crop giữa của bản ngang, vì crop làm mất carry và subject ở rìa.

Định nghĩa hoàn thành — Spec G

- `words[]` có trong dữ liệu job sau Phase 2; video mẫu đạt lệch highlight ≤ ±120 ms.
- Highlight là hàm thuần của t; seek tới giữa một từ cho đúng từ đó được tô sáng.
- Job `format: "9:16"` render ra MP4 720×1280 đúng tỉ lệ, qua Oracle framing ở viewport dọc.
- Job 16:9 cũ không đổi hành vi (regression test end-to-end pass).

8. Spec H — Font subsetting & comp tự chứa P1

Phase tác động: Phase 3 (sau khi ráp comp, trước khi lưu). **Mục tiêu:** file HTML của job thật sự tự chứa về typography — render headless ở worker nào cũng đúng font Space Grotesk / Inter của brand LIME, không phụ thuộc mạng, không lỗi "rớt font" khi render lại job cũ. Cách làm học từ ý tưởng subsetting của motion-web và tự cài đặt lại.

8.1. Quy trình subset (chạy ở cuối Phase 3)

1. **Chốt nội dung:** subset chỉ chạy sau khi mọi text của comp đã cố định (lời đọc, tiêu đề, chữ trên màn hình). Text sinh sau bước này phải qua lại bước subset.
2. **Quét glyph:** thu thập mọi ký tự phân biệt xuất hiện trong comp, cộng bộ dự phòng bắt buộc ở mục 8.2.
3. **Cắt font:** dùng `pyftsubset` (fontTools) cắt Space Grotesk và Inter theo tập glyph trên, xuất woff2.
4. **Inline:** nhúng font đã cắt dạng base64 trong `@font-face` ngay trong HTML; kiểm tra tĩnh xác nhận không còn URL font ngoài.

```
# Vi du lenh pyftsubset (tham so rut gon)
pyftsubset Inter.ttf \
  --text-file=glyphs.txt \
  --flavor=woff2 \
  --output-file=inter-subset.woff2 \
  --layout-features="kern,liga" \
  --unicodes="U+0000-00FF,U+0102-0103,U+0110-0111,\
U+1EA0-1EF9,U+20AB"
```

8.2. Lưu ý bắt buộc — tiếng Việt có dấu

> **Subset phải bao gồm đầy đủ dải Unicode tiếng Việt có dấu.** Đây là lỗi dễ mắc nhất: quét glyph từ một comp toàn tiếng Anh rồi tái dùng cho job tiếng Việt sẽ làm chữ có dấu rớt glyph và render thành ô vuông hoặc font thay thế. Biện pháp bắt buộc: tập glyph luôn gồm (a) glyph thực tế quét được, *cộng* (b) toàn bộ dải Latin Extended Additional (U+1EA0–U+1EF9, chứa các ký tự tiếng Việt có dấu thanh) và các ký tự tiếng Việt trong Latin Extended-A (ă, â, đ, ê, ô, ơ, ư và dạng hoa). Không được cắt bỏ phần (b) vì lý do dung lượng.

8.3. Test chấp nhận

1. **Render offline:** chặn toàn bộ network của phiên Playwright (không DNS, không request ngoài), render comp: typography phải đúng font brand, không fallback. Đây là test bắt buộc trong CI.
2. **Job tiếng Việt:** comp chứa đoạn văn tiếng Việt đầy đủ dấu (kể cả các ký tự ít gặp như "ữ", "ộ", "Đ") render đúng từng ký tự khi đối chiếu ảnh chụp với kỳ vọng.
3. **Dung lượng:** font inline của một comp nằm trong ngân sách cấu hình (gợi ý ≤ 300 KB cho hai họ font); vượt thì kiểm tra lại tập glyph, không bỏ dải tiếng Việt.

Định nghĩa hoàn thành — Spec H

- Bước subset chạy tự động cuối Phase 3; comp xuất ra không còn URL font ngoài.
- Dải Unicode tiếng Việt bắt buộc có trong cấu hình subset và có test riêng.
- Test render offline pass trong CI cho cả comp tiếng Anh và tiếng Việt.
- Render lại một job cũ (trước Spec H) vẫn hoạt động; job mới render lại sau 1 tháng cho typography khớp bản gốc.

9. Spec I — Template authoring spec P2

Phạm vi: quy trình đóng góp và chuẩn hoá motion template cho studio (hiện có hơn 50 template). **Mục tiêu:** kho template lớn lên mà chất lượng không loãng — mọi template đều có tài liệu, chạy độc lập được, và kiểm chứng được bởi Oracle (Spec C). Chuẩn này học từ ý tưởng "authoring spec" của motion-web và viết lại theo quy trình của LIME.

9.1. Chuẩn bắt buộc cho một template

Thành phần	Nội dung bắt buộc
`TEMPLATE.md`	Mô tả: template làm gì, dùng cho loại video nào; danh sách **params** (tên, kiểu, mặc định, khoảng giá trị) mà người dùng/LLM được tuỳ chỉnh; intent chuyển động diễn đạt bằng motion tokens của Spec A (không mô tả bằng tính từ chung chung)
`demo.html`	Demo chạy độc lập được ngoài studio: mở file là xem được template chạy với params mặc định; tuân thủ Deterministic Contract (Spec B) — có `__seekTo`, không `Math.random()`
Verify criteria	Danh sách assertions cụ thể để Oracle kiểm tra, ví dụ: "phần tử chính ở trong viewport tại mọi sample", "có ít nhất một carry mechanism khi ghép với scene mẫu", "không dùng easing ngoài token". Viết dạng kiểm chứng được, không viết "trông mượt"
Metadata	Format hỗ trợ (16:9 / 9:16 — xem Spec G), có dùng nhân vật procedural không (Spec F), phiên bản template

9.2. Badge "verified" trong studio gallery

Template chỉ hiển thị badge **verified** khi: đủ 3 thành phần ở mục 9.1, `demo.html` pass toàn bộ verify criteria trong lần chạy kiểm tra gần nhất, và pass Deterministic Contract lint. Badge là trạng thái suy ra từ kết quả kiểm tra lưu trong hệ thống, không phải cờ bật tay; template sửa code mà chưa chạy lại kiểm tra thì badge tự mất. Gallery vẫn hiển thị template chưa verified (kèm trạng thái rõ ràng) trong giai đoạn chuyển đổi, để không làm gãy các job đang dùng template cũ.

9.3. Lộ trình chuẩn hoá kho hiện có (hơn 50 template)

1. **Đợt 1:** các template được dùng nhiều nhất (xếp theo số job thực tế) — viết `TEMPLATE.md`, trích `demo.html`, viết verify criteria, chạy kiểm tra, gắn badge.
2. **Đợt 2:** nhóm template còn lại đang hoạt động; template nào thiếu element id cho Oracle (Spec C) thì bổ sung ở đợt này.
3. **Đợt 3:** template ít dùng hoặc trùng lặp: hoặc chuẩn hoá, hoặc đánh dấu deprecated trong gallery (vẫn render được cho job cũ, không cho chọn ở job mới).
4. **Template mới** từ thời điểm spec có hiệu lực: bắt buộc đủ chuẩn ngay khi nộp, không có ngoại lệ "bổ sung sau".

Định nghĩa hoàn thành — Spec I

- Mẫu `TEMPLATE.md` và bộ verify criteria mẫu được ban hành; quy trình nộp template mới cập nhật theo chuẩn.
- Badge verified hiển thị trong gallery và tự mất khi template chưa chạy lại kiểm tra sau sửa đổi.
- Đợt 1 chuẩn hoá hoàn tất với danh sách template cụ thể đính kèm báo cáo đợt.
- Không job nào đang chạy bị gãy vì chuẩn hoá (template cũ vẫn render được cho job cũ).

Phụ lục A. Bảng tra nhanh: spec → phase → ưu tiên → công sức

Công sức là **ước tính** theo cỡ S (vài ngày) / M (1–2 tuần) / L (trên 2 tuần) cho một kỹ sư quen codebase; phải ước lượng lại khi lập kế hoạch sprint.

Spec	Tên	Phase	Ưu tiên	Công sức (ước tính)
A	Motion Constitution + carry plan	1	P0	S–M
B	Deterministic Rendering Contract	3, 5	P0	M
C	Oracle QA (phase 4.5)	4.5 (mới)	P1	M–L
D	Shutter motion blur	5	P2	M
E	Sound from events	3, 6	P2	M–L
F	Nhân vật procedural (MIT)	1, 2, 3	P2	L
G	Karaoke subtitle + dọc 9:16	1, 2, 3, 5	P1	M
H	Font subsetting, comp tự chứa	3	P1	S
I	Template authoring spec	Quy trình	P2	M (theo đợt)

Phụ lục B. Ma trận license compliance

Nguyên tắc chung: ý tưởng và phương pháp không bị license chặn; code và tài liệu gốc thì có. Tài liệu này viết bằng lời văn riêng của LIME và không chứa code hay đoạn văn gốc của ba repo, ngoại trừ phần code MIT được phép ở Spec F.

Repo	License	Được làm	Không được làm
onetake	PolyForm Noncommercial 1.0.0	Học ý tưởng: carry, oracle và các ngưỡng đo, shutter blur, sound from events; tự cài đặt lại hoàn toàn (Spec A, C, D, E)	Sao chép code (thư viện moves, scripts), tài liệu hoặc assets vào LIME — LIME là sản phẩm thương mại
motion-web	CC BY-NC 4.0	Học ý tưởng: motion tokens, luật anti-slop, cách tổ chức verify, font subsetting, template authoring; tự cài đặt lại (Spec A, C, H, I)	Sao chép code, tài liệu tham chiếu hoặc assets vào LIME cho mục đích thương mại
doodle-anim	MIT	**Port và sửa code trực tiếp** vào LIME, kể cả thương mại (Spec B về ý tưởng contract; Spec F về code nhân vật)	Không có cấm đoán đáng kể; **bắt buộc** giữ thông báo bản quyền/license MIT và ghi nguồn (Spec F, mục 6)

Phụ lục C. Roadmap P0 → P1 → P2

Mức	Spec	Lý do xếp mức này
P0	A — Constitution + carry plan B — Deterministic Contract	Chi phí thấp nhất, tác động rộng nhất. A chủ yếu là schema + prompt + token, không cần hạ tầng mới nhưng đổi chất lượng cảm nhận của mọi video. B chủ yếu là chuẩn hoá + kiểm tra tĩnh trên nền `__seekTo` đã có, và là tiền đề cho render song song, retry và cache an toàn.
P1	C — Oracle QA (4.5) G — Karaoke + dọc 9:16 H — Font subsetting	Đều xây trên tài sản sẵn có: C mở rộng preview Phase 4 và cần A ổn định trước (oracle đo carry do A khai báo); G tận dụng BeatSync Phase 2 và mở kênh phân phối ngắn; H là bước khép kín ở Phase 3, nên làm trước khi số job và nhu cầu render lại tăng.
P2	D — Shutter blur E — Sound from events F — Nhân vật procedural I — Template authoring	Giá trị cao nhưng chi phí/phạm vi lớn hơn: D tăng chi phí tính toán mỗi frame; E cần xây sound engine và đổi khâu mix; F là một dòng hình ảnh mới; I là chuẩn hoá dần theo đợt. Riêng F có thể khởi động sớm dạng thử nghiệm vì code MIT đã sẵn và không phụ thuộc spec khác.

Thứ tự implement gợi ý: A và B song song → C (ngay khi schema của A chốt) → H và G trong cùng đợt P1 → D, E, F, I theo nguồn lực. Phụ thuộc cứng cần tôn trọng: C phụ thuộc A (carry) và B (seek ổn định); D làm leg curves của C đổi hành vi (fail thành warn) nên hai spec phải dùng chung cờ cấu hình; G và H không phụ thuộc nhau.

Phụ lục D. Nguồn

Ba repository gốc của tác giả feitangyuan, đối chiếu ngày 14/10/2026 — ghi nguyên văn URL:

- onetake — https://github.com/feitangyuan/onetake
- motion-web — https://github.com/feitangyuan/motion-web
- doodle-anim — https://github.com/feitangyuan/doodle-anim

Tài liệu tiền thân: “Báo cáo 3 repo motion & spec LIME” (14/10/2026) — chứa tổng quan ba repo, ma trận so sánh và bản phác thảo gốc của 9 spec. Các ngưỡng và tham số trong tài liệu này (carry score 0.5/0.75, 80 px/frame, −16 LUFS, −3.5 dBTP, 720×1280, stagger 0.04/0.08/0.15s) là giá trị khởi điểm cho v1: sau khi có dữ liệu job thật, team nên hiệu chỉnh và ghi lại quyết định hiệu chỉnh kèm ngày, thay vì coi chúng là hằng số bất biến.

> **Nguyên tắc khép lại:** thiết kế ở boundary (Spec A), nói cùng một từ vựng motion (Spec A), render lặp lại được tuyệt đối (Spec B), và không video nào tới tay người dùng trước khi một oracle đã đo nó (Spec C). Bốn spec còn lại mở rộng chất lượng và kênh phân phối trên nền đó.